Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 11

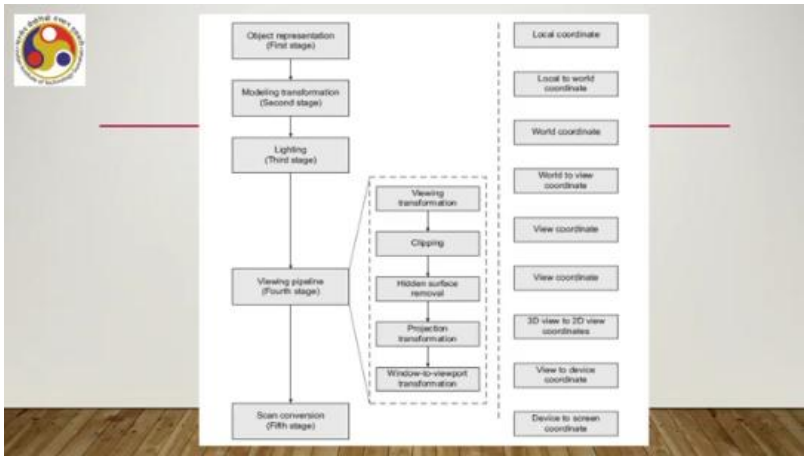
Lecture – 56

VR resource optimization

Hello and welcome to the NPTEL MOOCs course on Foundations for Virtual and Augmented Reality Systems. We are going to continue our discussion on resource optimization, specifically in the context of virtual reality. So, if we quickly recap what we have done so far, earlier we talked about the basic idea of computer graphics. Now, why are we talking about computer graphics? Because the key component of any immersive experience is the rendering part, how the scene is rendered in front of the eyes of the user on the display device, and the idea of rendering is covered in the discipline of computer graphics. So, it is essential to know the complexities involved in rendering, and in order to do that, we need to know or at least we should have some idea of computer graphics. So, we started our discussion on computer graphics. We have gone through some basic terminologies.

Then, after covering the basic idea, we covered the idea of the 3D graphics pipeline and also the stages constituting the pipeline. We have also covered the first stage of the pipeline, that is, the object representation. Note that although we are talking about this as the first stage, in object representation, there is no computation as such related to intensity calculation or determination of the pixel intensities. So, we may or may not call it the first stage, but for the sake of simplicity, we are calling it the first stage of the graphics pipeline. Now, in this lecture, what we are going to do is cover the second and third stages of the graphics pipeline.

So, two more stages will be covered in this lecture. But before we do that, let us quickly have a relook at the 3D graphics pipeline as discussed in the previous lecture. So, we have these five stages: object representation, first stage. which happens in the local coordinate system. We have already talked about this one.



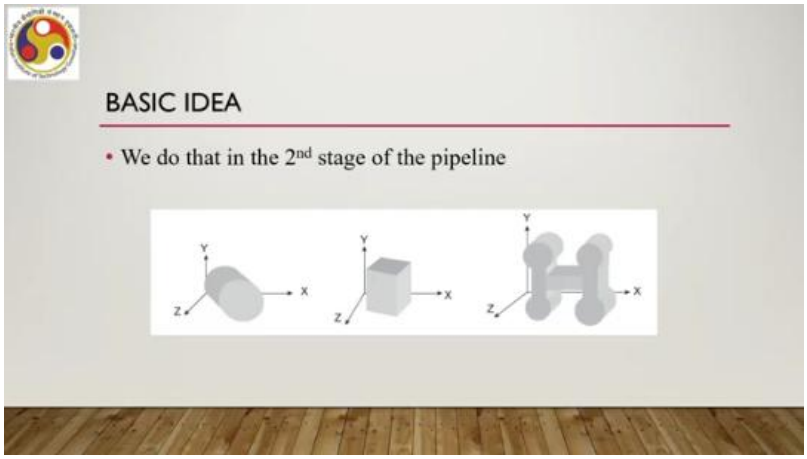
The second stage is modeling transformation, in which the local coordinate description is transformed to a world coordinate description; that is, local to world transformation, local to world coordinate transformation. Third is the lighting stage. which takes place in the world coordinate reference frame. Then we have the fourth stage, that is the viewing pipeline, containing several sub-stages starting with viewing transformation, where again we transform from world coordinate system to a 3D view coordinate system. This is followed by clipping, an operation taking place in the 3D view coordinate system, followed by hidden surface removal, another operation taking place again in the 3D view coordinate system. Then we have projection transformation, where we transform from 3D view coordinate to 2D view coordinate system, and then we have window to viewport transformation, where we transfer the scene description in the 2D view coordinate system to a device coordinate system. The fifth and final stage is scan conversion, where we transfer or transform from the device coordinate scene description to screen coordinate or pixel grid description of the scene.

So, these are the five stages. Out of these, we have already covered the first one, and we are going to cover these two stages, namely modeling transformation and the lighting, or the second and third stages, in this part of the lecture. So, let us start with stage 2, that is modeling or geometric transformation. So, both terms are used; either modeling transformation can be used, or we can use the term geometric transformation.

Now, what is the basic idea behind this stage? Earlier, we talked about object representation. Now, these object representation techniques allow us to represent objects individually. So, for each object, we can have an individual definition of the object, and that definition is often called the local or object coordinate system description of the object. So, essentially, we can think of each object as having its own coordinate reference frame in which the object is defined; that is, its position, dimension, and so on are defined.

Now, in order to construct a scene, what we need to do is put multiple objects together. Now, each individual object is defined in its own coordinate system, or its local coordinate system. We need to redefine the object in a common coordinate system where all the objects can be put together and defined. So, objects need to be assembled together in a different setting, which we can call the scene or world coordinate system. So, essentially, what we do is basically

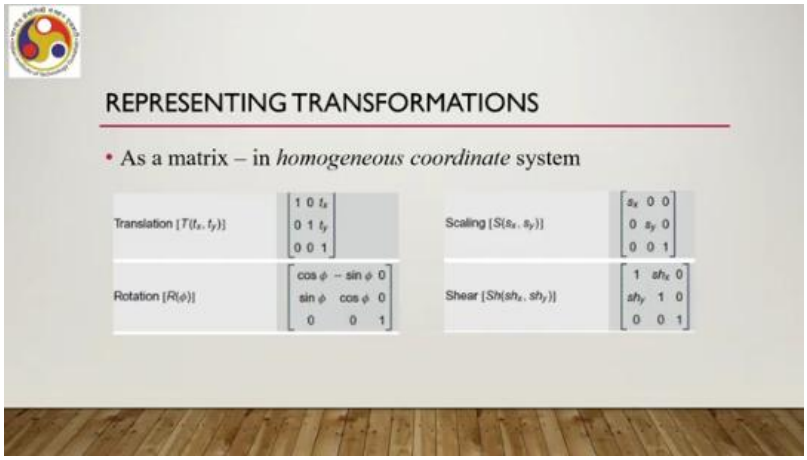
taking the object from its local coordinate description to a scene or world coordinate description. The idea is illustrated here. So, suppose we have object 1 and object 2, each having its own coordinate system as shown here. Now, using these two objects, we are constructing a scene which comprises this shape resembling the English letter H. So, as you can see, in order to construct this H, we have used multiple instances of object 1 and object 2 and that



too in different shapes, sizes, orientations, and so on. Say, for example, this is one instance of object 1, a second instance of object 1, a third instance of object 1, a fourth instance of object 1. So, these are all object 1 and these are all object 2. So, there are four instances of object 1 and three instances of object 2 used to create this particular shape, and if you compare the definition to the final position or shape or orientation of the objects, you can see a lot of changes. So, this is what we are calling transformation from local coordinate definition to world coordinate definition or scene coordinate definition. So, this modeling transformation, the second stage, implies applying some operations on the object definition in local coordinates to transform to the world or scene coordinate system. All these operations can be derived from a set of basic operations. which are the basic transformations.

What are those basic transformations? We are all familiar with those transformations, such as translation, rotation by some angle either in a clockwise or anticlockwise direction around an axis, scaling, that is reduction or increase in the size of the object, and shearing, that is changing the shape of the object. So, These four constitute the basic transformations, and we can use combinations of these four to create any sort of transformation required to construct a scene.

Now, typically, each transformation is represented in the form of a matrix. But the matrix representation of transformations happens in a different setting, that is typically called a homogeneous coordinate system. So, for example, for a 2D transformation, that is a transformation happening in a 2D plane, we can have 3 by 3 matrices. For a 3D transformation, that is a transformation happening in 3D, we can have 4 by 4 matrices and so on. So, one extra dimension is added in the homogeneous coordinate system.

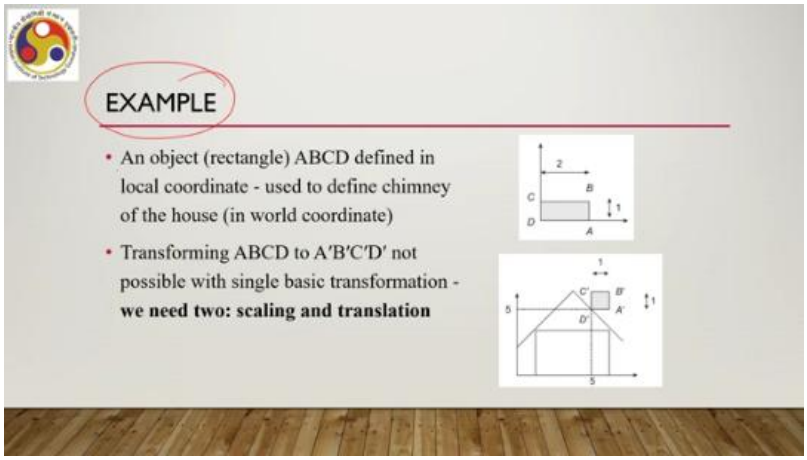


So, for example, for translation, we can have this 3 by 3 matrix for 2D translation, where T_x and T_y are the amounts of translation along the x and y directions. For rotation in 2D, we have this 3 by 3 matrix; ϕ is the angle of rotation. For scaling in 2D, we can have this 3 by 3 matrix, where Sx and Sy are scaling factors along the x and y directions. For shearing in 2D, the corresponding 3 by 3 matrix is shown here.

Here Sx, Sh_x and Sh_y are the shearing factors along the x and y directions. Similarly, we can have the matrix representation of transformations for 3D operations. Now, the matrix representation is a convenient way to represent transformations, and using these matrices we can transform points from an object's own local coordinate system to a world coordinate system. So, how does it work?

Given a point (x, y), we can assume that it is in the local coordinate system where the object is defined. We can represent it as a column vector. Then we can multiply the transformation matrix with the column vector to get the new point in the form of another column vector, P', which we can obtain using the transformation matrix S and the original point P to get the new point. So, effectively, the transformation refers to a matrix multiplication of the points which are on the surface of the object to the new points that ideally should be in the world coordinate system.

There is one issue, though: sometimes to construct a world coordinate scene, we need to perform multiple geometric or modeling transformations on an object. That is, not only the basic operations, but multiple operations may be required, and there we need to go for a composite transformation matrix rather than basic transformation matrices like translation, rotation, scaling, or shearing. Just to illustrate the idea, or rather the issue, let us consider one example: an object, which is a rectangle shown here, ABCD, is defined in the local coordinate system which is used to define a chimney of the house in the world coordinate system.

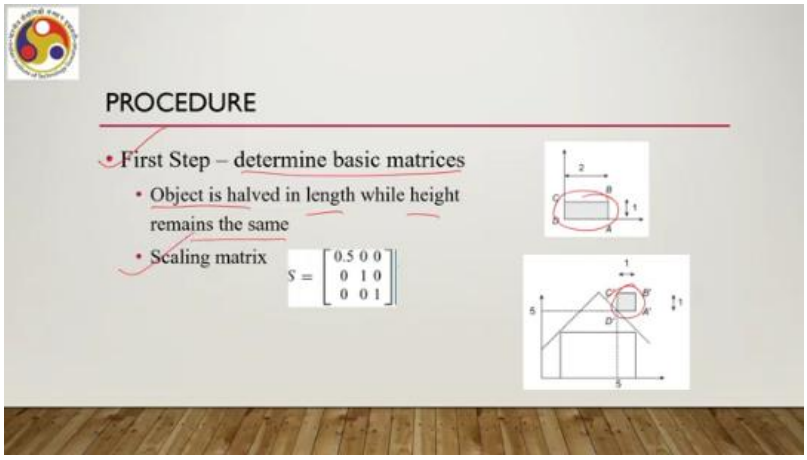


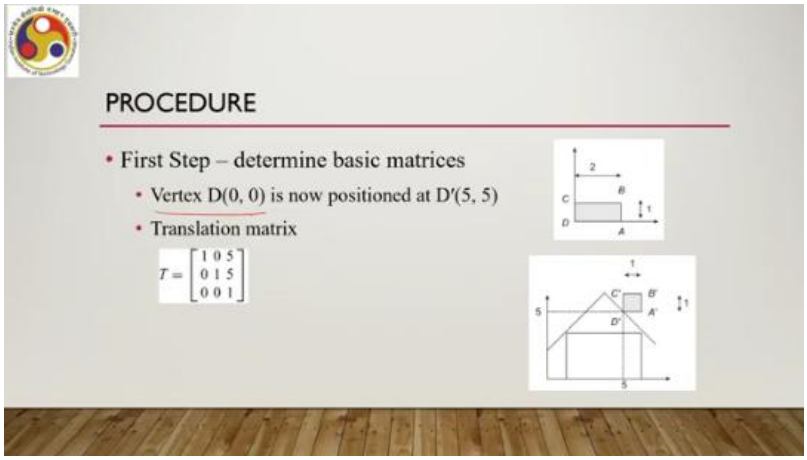
So, this rectangle is used to define a chimney-like shape in this house shown in the world coordinate scene. So, in order to do that, in order to construct this scene with the chimney in its place, What do we need to do? What we need to do is basically transform ABCD, the original rectangle in its local coordinate scene, to $A'B'C'D'$, that is this chimney-like shape. Now, that is not possible with only a single basic transformation.

So, using either translation only, rotation only, scaling only, or shearing only, we will not be able to do this. We will not be able to transfer or transform ABCD to $A'B'C'D'$. We actually require two basic transformations together. One is scaling and one is translation.

So, in order to perform the transformation, we need to go through stages. The first stage is to determine the basic matrices for individual transformations, that is, scaling and translation. As we can see, from here to this shape in the world coordinate scene, the object is halved in length while the height remains the same. That means the scaling factor along the x direction must be half, and the scaling factor along the y direction should be 1.

So, the scaling matrix should be of this form (0.5 0 0, 1 0 0, 0 0 1) where Sx equals 0.5, the scaling factor along the x direction, and Sy equals 1 since there is no change. Also, we need to translate. So, vertex-wise, if we look, we can see that vertex D(0, 0), that is the origin, has been translated to D', that is 55 in the world coordinate scene.

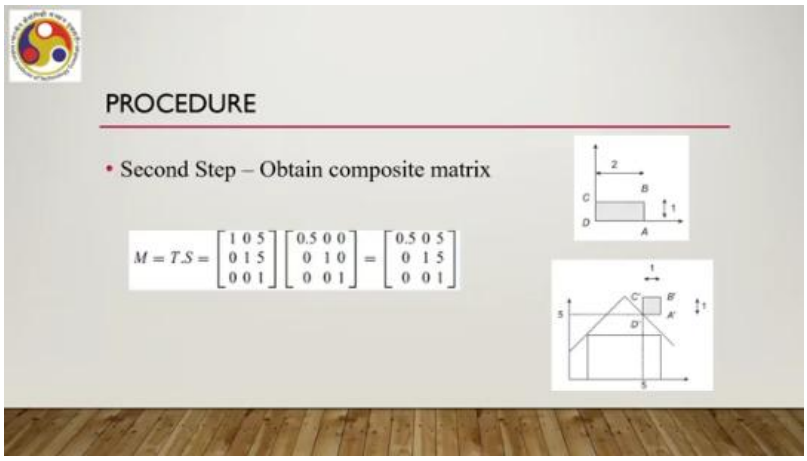




So, the translation matrix should be this one: (1 0 5, 0 1 5, 0 0 1) , that is Tx equals 5, Ty equals 5. Translation along the x-axis should be 5, translation along the y-axis should be 5. So, now we have identified the two basic matrices. So, the second step is to obtain the composite matrix, the composite transformation matrix where both are used. So, the first transformation applied to the object is the rightmost in the sequence.

The next transformation is placed on the left. This is how we can obtain the composite matrix. And we continue in this way until the last transformation. Since we have only two, after the first two steps, we can stop. That is, the first transformation, whatever it is, we put it on the right-hand side.

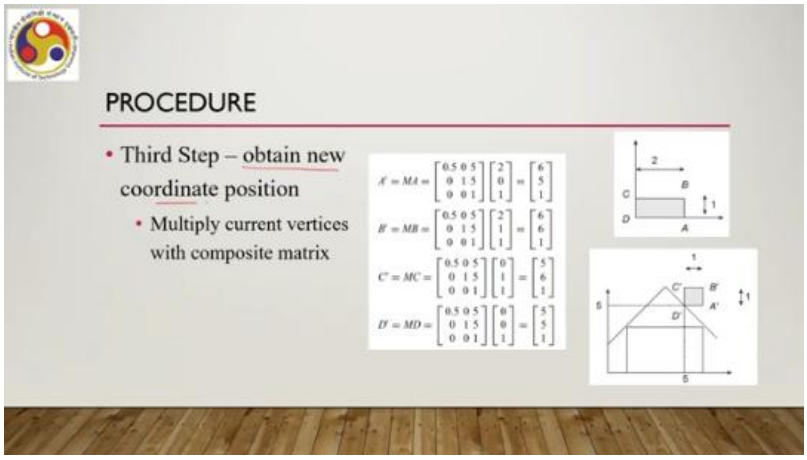
That is T1. Then the next transformation is placed on the left of this transformation. So if the next one is T2, then we put it in this way. So right-to-left arrangement of transformations that are applied. And then we perform a matrix multiplication of those transformations.



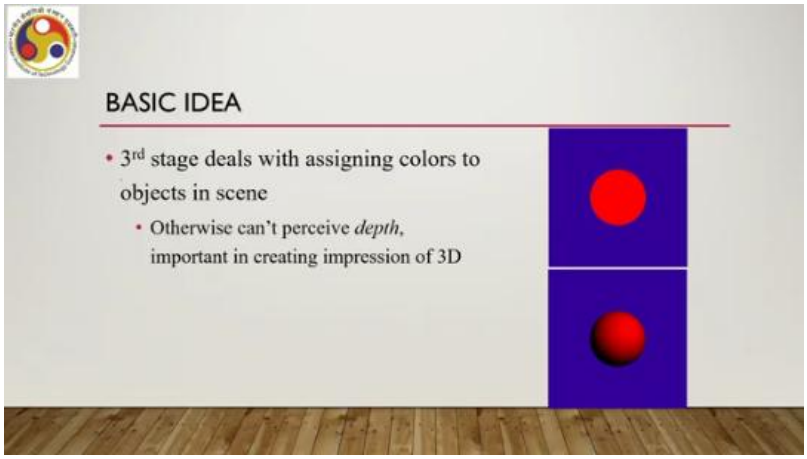
So if we do the same for our example, We can obtain the composite matrix by multiplying these two: translation followed by scaling. So, right to left, then we multiply these two. So, if we follow the same, then we can see that scaling is the first transformation applied, followed by translation, and we arrange them from right to left as mentioned previously.

And then we take a multiplication, we perform a multiplication of the matrices to get a composite transformation matrix in this form; that is the composite transformation matrix.

In the third step, what we do is obtain the new coordinate positions by multiplying the current vertices with the composite matrix. Current vertices are given here: A, B, C, D. The composite matrix is this one. Current vertex, if it is A, then A dash would be given by multiplying M with A. Similarly, multiplying M with B we get B dashed as shown here; C dashed in a similar fashion we can get, and D dashed similarly we will get like this, by multiplying M with current vertex D. So, this is how we can obtain the transformed points.



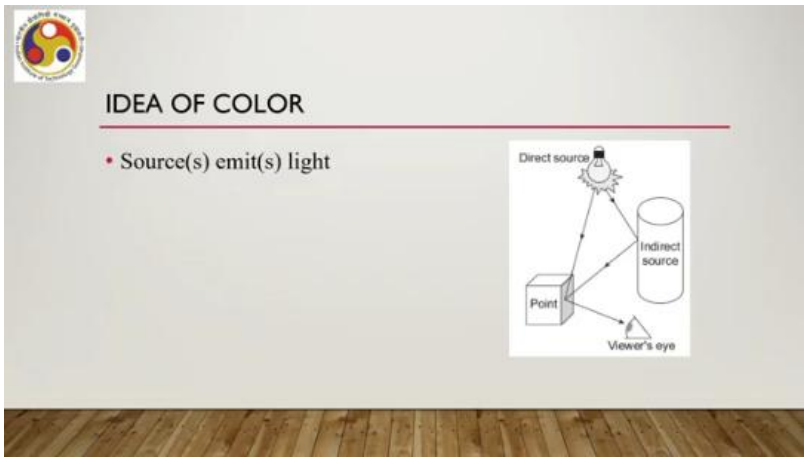
in the world coordinate scene. One thing we should note here is that matrix multiplication is not commutative, and the sequence is very important. So, if we are unable to formulate the sequence in the proper order, like right to left as we have discussed previously, then if the sequence is wrong, it may lead to an incorrect result. So, the order in which we put the transformations before multiplying them is very important. Nevertheless, without going into further details of all the complexities involved in modeling or geometric transformation, we can say that modeling transformation involves multiplication of a transformation matrix, whether it is a basic matrix or a composite matrix, with the vertices to get a transformed vertex in the world coordinate scene. That is about stage 2. Let us talk about stage 3, that is, color computation. So, in the third stage, what we do is basically



we try to assign colors to the objects in the scene. If a color is not assigned, then we will not be able to perceive depth, which will affect our perception of the 3D scene. So, essentially, the quality of experience will reduce, as shown in this example. The top one is without assigning proper color. Some color is there, but it is not done properly, and the bottom figure shows if we assign colors properly, then we can get a more realistic perception. Now, in order to do that, we use lighting or shading models. The term lighting model and the term shading model are distinctly used in the context of computer graphics, although they may physically represent more or less similar things. But here the terms are differentiated in terms of the computation involved. Lighting models are those that involve more computation as compared to shading models.

Shading models are used to reduce computation but without compromising on the quality of experience. Here, let us focus our attention on lighting models. Shading models are essentially approximations of color assignment, approximations of methods used to assign colors using the lighting models. So, they also rely on lighting models. So, let us try to understand the lighting model that will give a holistic understanding of this color assignment process.

So, lighting models compute colors in terms of continuous intensity values; essentially, these are real numbers typically within the range 0 to 1. So, typically a normalized real number. Now, these real numbers are mapped to computer-supported discrete values. So, we will come back to this point at the end of this lecture. In order to understand the lighting model, let us first try to quickly understand the idea of color.

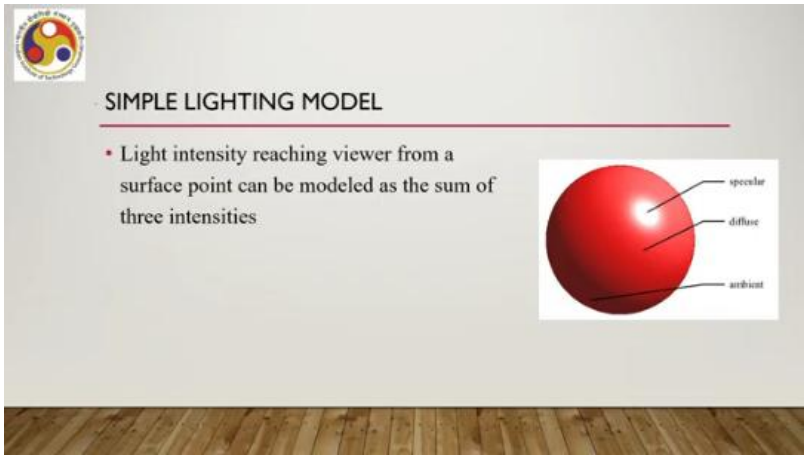


Now, a source emits light. Say, for example, this is a scenario: here there is a direct light source; it emits light. The emitted light energy falls upon a point on the surface, say here. Sometimes, emitted light falls on the point after getting reflected from surrounding surfaces. So, this light, after getting reflected from here, comes and falls on the same point. The incident light then reflects.

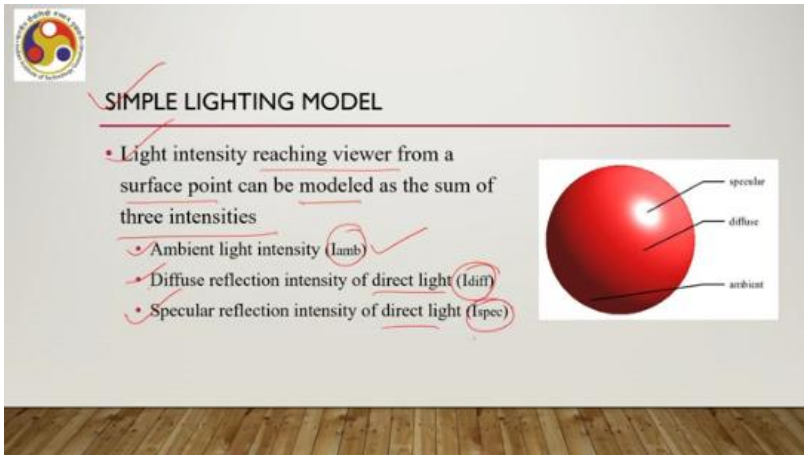
So, whatever light is incident at this point gets reflected and comes to our, or the viewer's, eye. The intensity of light reaching the viewer's eye results in color perception. Whatever the intensity of this reflected light is determines what color the viewer is going to perceive. Now, in computer graphics, we are interested in simulating this process through a lighting

model. So, to compute the intensity of this reflected light, essentially the color of the surface point from where the light is coming,

we simulate the process using a lighting model in computer graphics. Now, there are many such models. Let us understand the process. Let us go through one simple lighting model. In this model, the light intensity reaching the viewer from a surface point can be modeled as the sum of three intensities.



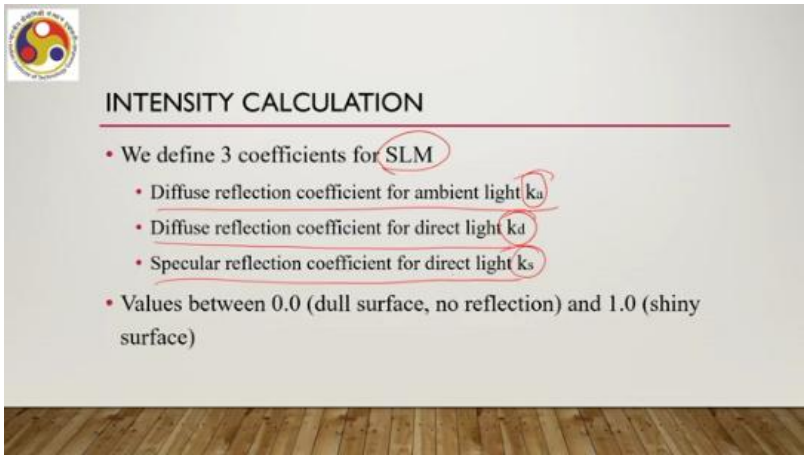
Ambient light intensity. which you can represent with I amb, then diffuse reflection intensity of direct light I diffuse, I diff and specular reflection intensity of direct light that is I spec. The idea is illustrated here. So, whatever light falls on the surface of this object after getting reflected from surrounding surfaces that is called ambient light. If light is coming directly from the source and getting reflected in a diffuse manner then that is the diffuse light component.



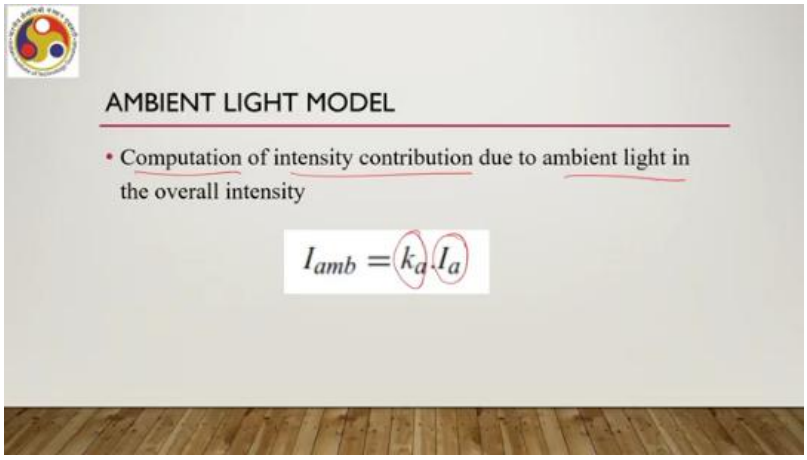
If the light comes from the source and gets reflected with a specular effect that is a shiny bright spot effect then that is the specular reflection component. So, each type of light will have its own intensity and these three intensities together after combining these three intensities they come to the viewer's eye. And correspondingly we can have the simple lighting model which says that the intensity of the incident light on the viewer's eye is the

sum of the incident light due to ambient component, due to diffuse component and due to specular component.

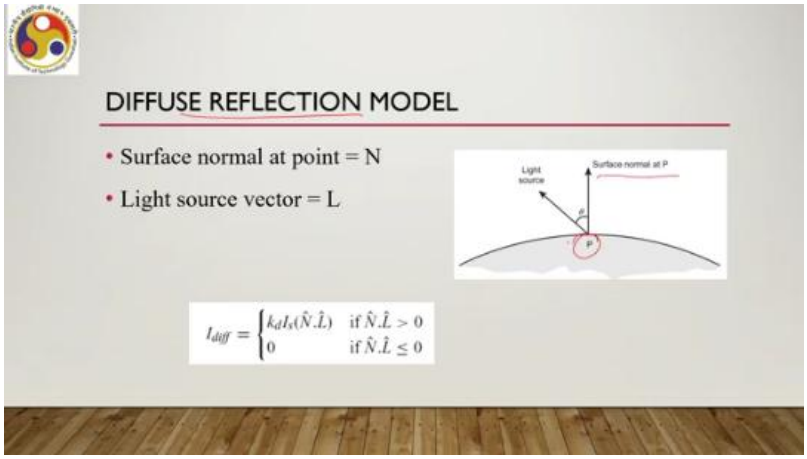
This is the simplistic way of looking at the lighting model. Now, the reflected light intensity is a fraction of the incident light intensity. Whatever falls, a fraction of it gets reflected and comes to the viewer's eye. Now, this fraction is determined by a surface property known as reflection coefficient or reflectivity. We define three coefficients for the simple lighting model corresponding to three types of incident lights that is diffuse reflection coefficient for ambient light K_a, diffuse reflection coefficient for direct light K_d and specular reflection coefficient for direct light K_s. Now, the values of each of these reflectivity or reflection coefficients vary between 0.0 to 1.0; that is, if the value is close to 0 or 0, then it represents a dull surface with no reflection, as most of the light gets absorbed by the surface. And if it is close to 1 or 1, that means a shiny, mirror-like surface where all light gets reflected. So, if we want to generate different effects



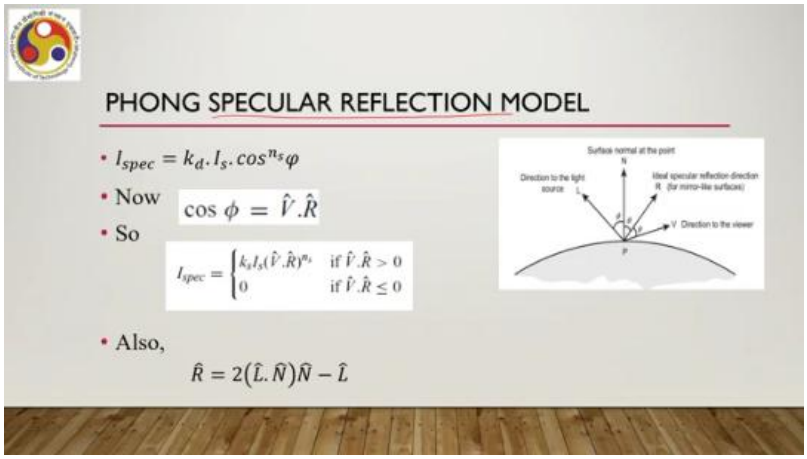
between dull and shiny surfaces, we can vary these coefficients accordingly. Now, to compute the intensity contribution due to the ambient light, what we need to do is simply multiply the ambient light intensity falling on the surface point with the reflectivity for the ambient light, that is Ka, to get the fraction of the ambient light that comes to the viewer's eye. So, this is $I_{amb} = k_a . I_a$



For the diffuse reflection component, it is slightly more complicated. Here, we need to take into account the surface normal at the surface point from where the light gets reflected and the light source direction. So, let the surface normal be n, a vector, and the light source direction be L, another vector. Then we can compute the diffuse reflection component coming to the user's or viewer's eye as K_d Is, where Is is the source light intensity and the dot product of these two vectors, N.L, if N.L is greater than 0; otherwise, it is 0, which means there is no reflection.



Now, specular component calculation is even more complex. Here we are going to talk about one approximate model. named after its inventor Phong. So, the component is I_{spec} equal to $K_d.I_s.\cos^{n_s}\phi$. ϕ is, as you can see in this figure, the angle between the viewer direction and the ideal specular reflection direction; that is, if it is a mirror, then the direction of the reflected light.

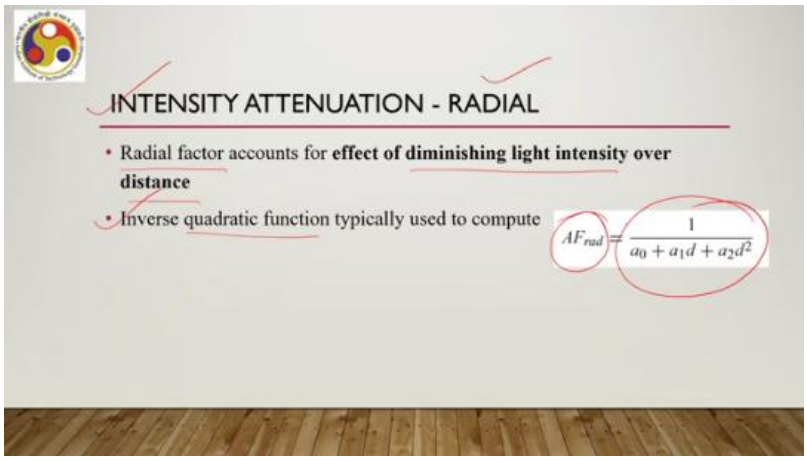


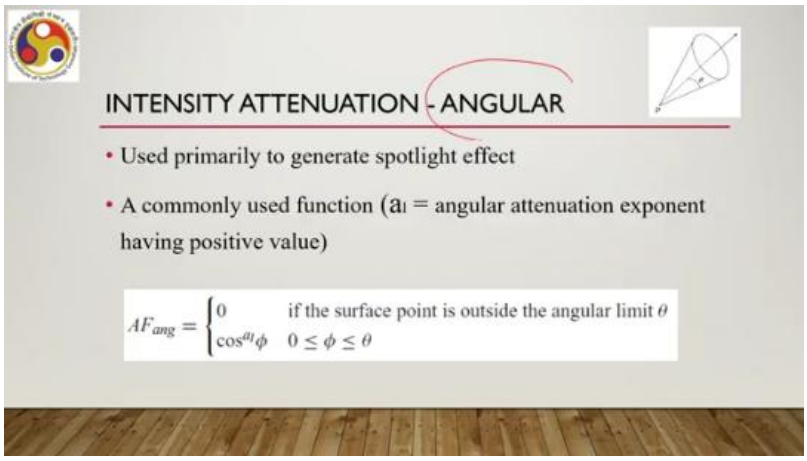
Now, we can see from here that $\cos\phi = V.R$, another dot product of the two vectors. V is the viewer direction vector, R is the ideal specular reflection direction vector. So, we can say I_{spec} is $K_s.I_s$, where Is is the source light intensity and Ks is the reflectivity for the specular component. $(V.R)^{n_s}$ if V dot r is greater than 0, otherwise it is 0. Now, this r, the ideal specular reflection direction, we can represent with the surface normal at that point, the normal vector at that point that is N, and

the light source light direction vector L as r equal to $2(L.N)N - L$. So, we can replace R here with this expression to finally come up with the component as a dot product of the vectors L, N, and V. So, we need to know these vectors in order to be able to compute the specular component. Now, whatever we have discussed so far assumes that no matter where the light source is, if its emission intensity, if the intensity of the emitted light at the source is IS, then the same intensity will come to the surface point irrespective of the distance between the source and the surface point.

But in practice, that typically never happens. Typically, light intensity diminishes. over distance. So, we need to take care of this factor that is intensity attenuation over distance. So, there are two ways to take care of it: one is radial intensity attenuation, that is, the radial factor accounts for the effect of diminishing light intensity over distance.

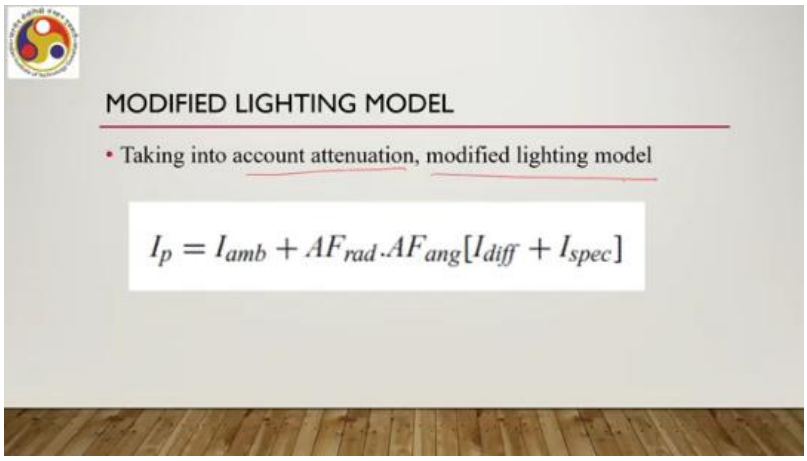
Typically, an inverse quadratic function is used to compute this factor in the form f, that is, attenuation factor for radial attenuation, as given by this expression inverse quadratic function where d is the distance between the source and the surface point; a0, a1, a2 these are coefficients varied to produce a better realistic effect. So, we can control these values. Also, angular intensity attenuation can be taken care of, can be considered primarily if we are interested in generating spotlight effects such as shown here.





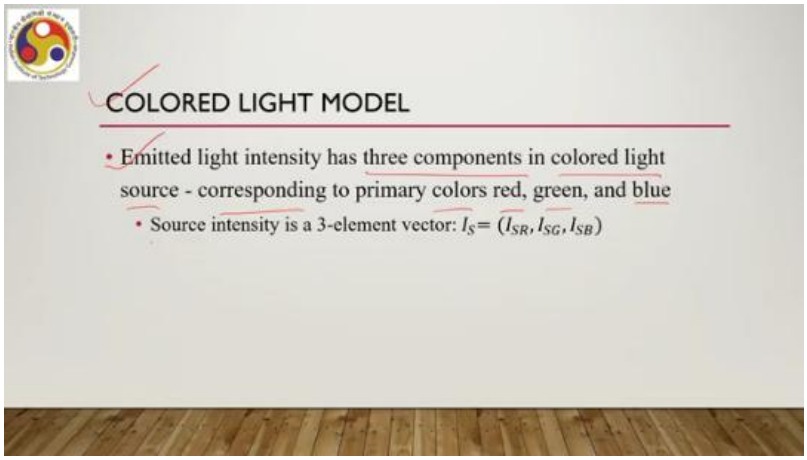
If we want to have an effect like this, where within a region only those elements will be visible or brightly visible and outside that spotlight zone things will be darker. If we want to generate such type of effect, then we can make use of intensity attenuation factor. A commonly used function to determine the factor is this given here: attenuation factor angular. It is 0 if the surface point is outside the angular limit theta, that is, the range of the spotlight cone. And if it is within that angular limit, then it varies in the form $\cos^{a1} \phi$ where a1 is the angular attenuation exponent, having a positive value, and we can vary this value to achieve different effects. So, this is for angular attenuation. So, if we take into account this attenuation factor, that is, the light source intensity changes based on the distance of the surface point from the light source. Then, taking into account this attenuation, we can modify the simple lighting model.

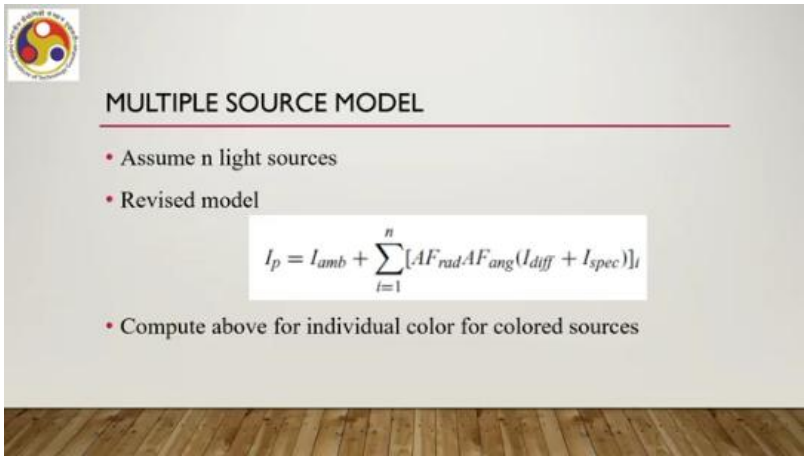
Earlier it was simply $I_m + I_d + I_{spec}$. Now we can say I_m; here we are not taking into account the intensity attenuation, but for diffuse and specular reflection, we are doing that by multiplying these two factors with the sum of these two components. Now, if these are one, that means the factor is uniform, meaning we are not taking them into account; then it boils down to the earlier formulation. If it is not one, then the effect is taken care of.



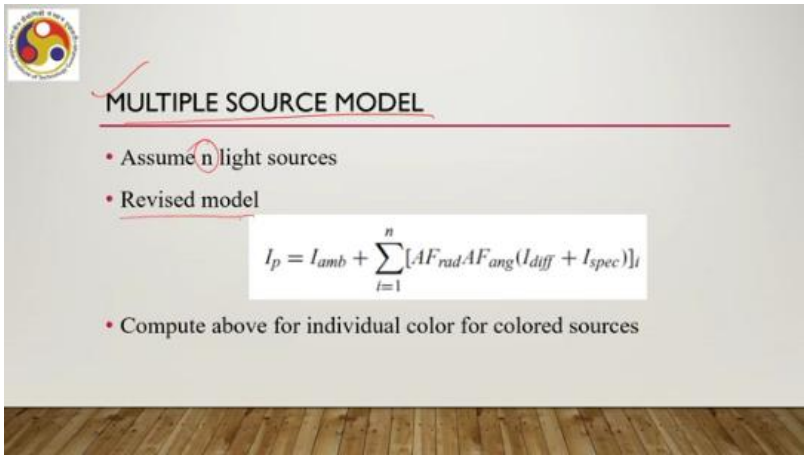
Also, when we are thinking of assigning colors, so, the colored light model is a slightly different way. Emitted light intensity has three components in such a case, rather than a single component; in a colored light source, these correspond to the primary colors red, green, and blue. Now, these three intensities combine together to give the perception of any color.

So, the source intensity is a three-element vector I_s: I_{sr}, I_{sg}, I_{sb}. corresponding to the three primary colors. Reflection coefficients also need to be specified as three-element vectors: k equal to one for red, one for green, one for blue; kd again one for red, green, blue; and ks one for red, green, and blue. So, when we compute color, we have to individually compute these





components, and then once these three are generated together, we perceive the color. Now, so far we have talked about a single light source. What happens when there are multiple light sources? Let us assume that there are n light sources. So, in such a case, we need to further revise the model.



It will $I_p = I_{amb} + \sum_{i=1}^{n} \left(AF_{rad} AF_{ang} \left(I_{diff} + I_{spec} \right) \right)_i$, that is, for each light source we have to calculate this part, and then we have to add those up, and each of these we need to

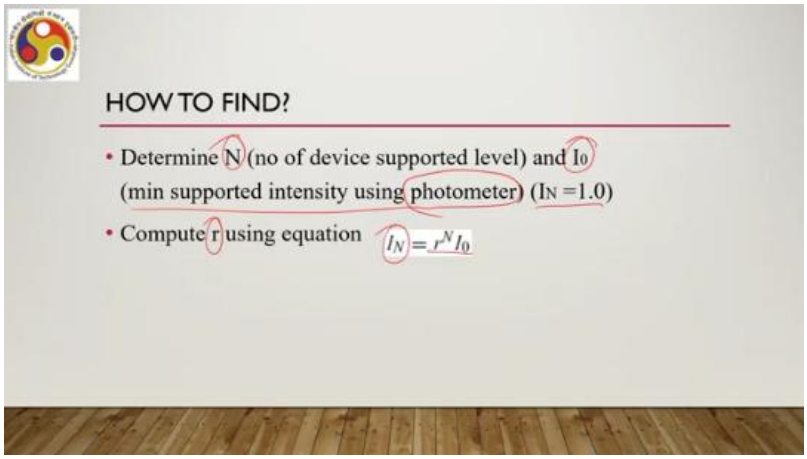
compute for individual colors for the colored light sources. So, if we have multiple colored light sources, then this needs to be done for individual colors for the colored light sources.

So, you can get some idea of the kind of computations involved in assigning colors. In the previous case, we have seen the computation involved in transformation. Now, we are seeing the computation involved in assigning colors. The other part is intensity mapping. So, the lighting model computes an intensity value for a pixel, but then that value can be anything between 0 and 1.

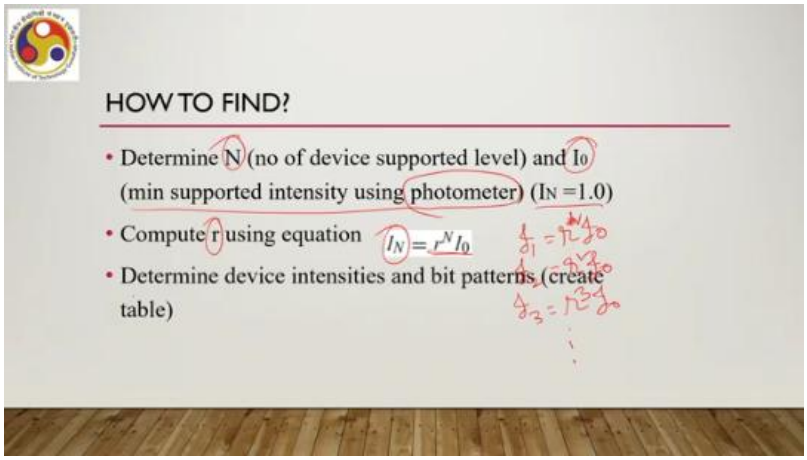
An infinite range. However, devices do not support arbitrary ranges; they have a finite range of values. Say, for example, an 8-bit display will support 2 to the power 8 or 256 colors for each of the pixels. So, we have 256 values supported by the device. But then,

the range 0 to 1 is infinite; any value can be generated. So, what we need to do is we find the nearest value to I_p from the device-supported intensity values, say I_k, then the frame buffer stores the bit pattern for I_k at the pixel location. So, how to do that? Essentially, what we need to know, what we need to do, is determine the n number of device-supported levels. Then we need to determine the minimum device-supported intensity value I_0 using a photometer, and we know that the maximum value can be 1. So, we have $I_0$and 1 known, and n, the number of levels supported.

Then there is a factor r which we need to compute using the equation I n equal to I r to the power n I naught. Now, this equation is formed based on one of our psychophysiological properties, that is, we can perceive relative change between two intensity values rather than absolute change in the intensity values. Based on that, we can formulate this equation as successive intensity values to be represented by this.



$i_1 = r^N I_0$, $r^1 I_0$, $i_2 = r^2 I_0$, $i_3 = r^3 I_0$ and so on up to the nth value. Now, once R is determined, we need to calculate these values I0, I1, I2, I3, store that in a table, and for each such value we assign a bit pattern and also store that along with those values. Then we determine the device intensities and bit patterns and store them in a table, and finally, when we compute using a lighting model pixel intensity, we map the computed intensity to the nearest device intensity. So, we have a table storing I0, I1, I2, IN and corresponding bit patterns, bit pattern 1, bit pattern 2, bit pattern 3,



n. So, once we have the computed value, we map it to the table; that means we map it to the nearest intensity, and the corresponding bit pattern is obtained and stored in the frame buffer. That is how we map a computed value to the device-supported value. So, let us summarize what we have covered so far. In this part of the lecture, we have covered two more stages of the pipeline: modeling transformation and intensity calculation lighting.

In modeling transformation, we have seen that to transform a point from its local coordinate system to the world coordinate system, we first need to obtain the transformation matrix, which can be an individual basic matrix or a composite matrix. A composite matrix is more common, and then we multiply the point vector with the composite matrix to determine the transformed point. That is one set of computations. Once the point is transformed in the world coordinate scene, then for that point, we assign some color; that is, we compute the intensity of the light which gets reflected from that point and comes to the user's eye. Note that here we are not actually computing color at the pixel level, but color in the 3D scene at the point level. And in order to do that, we need a lighting model. We have quickly gone through a simple lighting model to get some idea, and we have also talked about how to map the color computed with the lighting model to a color supported by the device.

The mapping table is device-specific. For a given display, the table is fixed, and we need to map it to the supported value. In the next part of the lecture, we are going to cover the remaining stages of the pipeline. I hope you could follow the topics and understand the concepts covered here. For more detail, of course, we are keeping things at a very broad, simple level. For more details, you can always refer to the material referenced at the end of this lecture.

So, looking forward to meeting you in the next part of the lecture. Thank you, goodbye, and happy learning.